



CS 202 Database Systems

Group Project Report

Online Food Ordering System

Group ID: 6

Arda Cirit S033849

Efe Atçalı S034150

Instructor: Hüseyin Ulusoy

DEPARTMENT OF COMPUTER SCIENCE

Spring 2026

May 31, 2026

Contents

1	Introduction	3
1.1	System Overview	3
1.2	Three-Layer Architecture	3
1.3	Technology Stack	4
1.4	Report Structure	5
2	Entity–Relationship Diagram	5
2.1	Entities	5
2.2	Relationships and Cardinalities	6
2.3	Note on a Design Revision	7
3	Functional Dependencies	8
3.1	Users	8
3.2	UserAddress	8
3.3	UserPhone	8
3.4	Restaurant	8
3.5	RestaurantKeyword	8
3.6	MenuCategory	8
3.7	MenuItem	9
3.8	Coupon	9
3.9	Order	9
3.10	OrderItem	9
3.11	Rating	9
4	Normalization	10
4.1	First Normal Form (1NF)	10
4.2	Second Normal Form (2NF)	10
4.3	Third Normal Form (3NF)	10
4.4	Deliberate, Documented Denormalizations	11
5	Key Design Decisions and Constraints	11
5.1	Storage Engine, Initialization, and Sample Data	12
5.2	Primary, Unique, and Not-Null Constraints	12
5.3	ENUM and CHECK Constraints	12
5.4	Foreign Keys and ON DELETE Behavior	12
5.5	The Circular Users ↔ UserAddress Reference	13
5.6	Composite Foreign Key on MenuItem	13
5.7	Application-Level Business Rules	14
5.8	Password Security	14
5.9	Centralized Error Handling	15
6	Main SQL Queries	15
6.1	Restaurant Search with Relevance Ranking	15
6.2	City-Filtered Browse	16
6.3	Order Placement (Two Inserts)	16
6.4	Guarded Status Transitions	16
6.5	Monthly Statistics	17

6.6	Coupon Resolution	18
6.7	Login and Salt Retrieval	19
6.8	Parameterized Statements and Injection Prevention	19
7	Implemented Functionalities	19
7.1	Account and Session Management	20
7.2	Customer Functionalities	20
7.3	Manager Functionalities	22
7.4	System-Wide Services	24
8	Conclusion	24

Abstract

This report documents the design and implementation of an Online Food Ordering System, a full-stack application. The system lets customers register, browse and search restaurants in their own city, build an order from a restaurant's menu, apply discount coupons, track the order through its lifecycle, and rate a restaurant after delivery. Restaurant managers can create and manage restaurant profiles, organize menus into categories, define time-limited coupons, accept and fulfil incoming orders, and view a monthly sales summary. The application is built on a relational MySQL 8 database accessed exclusively through hand-written SQL over Java Database Connectivity (JDBC); a Java Spring Boot backend exposes the application logic as a REST (Representational State Transfer) interface, and a JavaFX desktop client provides the user interface. No Object-Relational Mapping (ORM) framework is used — every query is written manually. This document presents the Entity-Relationship (ER) diagram, the functional dependencies, the normalization of the schema up to Third Normal Form (3NF), the key design decisions and integrity constraints, the main SQL queries, and the implemented functionalities.

1 Introduction

1.1 System Overview

The Online Food Ordering System is a desktop application that connects customers with restaurants in their city. It reproduces the core experience of commercial delivery platforms: a customer registers, selects a delivery address, browses or searches the restaurants available in that city, builds a cart from a restaurant's menu, optionally applies a discount coupon, places the order, follows it through its lifecycle, and finally leaves a rating once the order is delivered. On the other side of the platform, a restaurant manager registers restaurants, organizes their menus into categories and items, defines coupons, processes the orders that arrive, and reviews a monthly sales summary.

The system supports two user roles, both stored in a single **Users** table and distinguished by a **role** attribute:

- **Customer.** Registers and logs in; manages multiple delivery addresses and phone numbers; browses and searches restaurants filtered to the city of the currently selected address; views menus; builds a cart; previews and applies coupons at checkout; places orders; tracks active orders; views completed order history; and rates a restaurant within 24 hours of delivery.
- **Manager.** Logs in; creates, views, and edits restaurants; manages menu categories and items; creates and deactivates coupons; views incoming orders and advances them through the lifecycle; views customer ratings; and consults a monthly sales statistics summary.

1.2 Three-Layer Architecture

The application is built as three distinct tiers that communicate over well-defined boundaries, as shown in Figure 1. This separation is a deliberate design goal of the project: the user interface, the application logic, and the database each live in their own layer with no leakage of responsibility across boundaries.

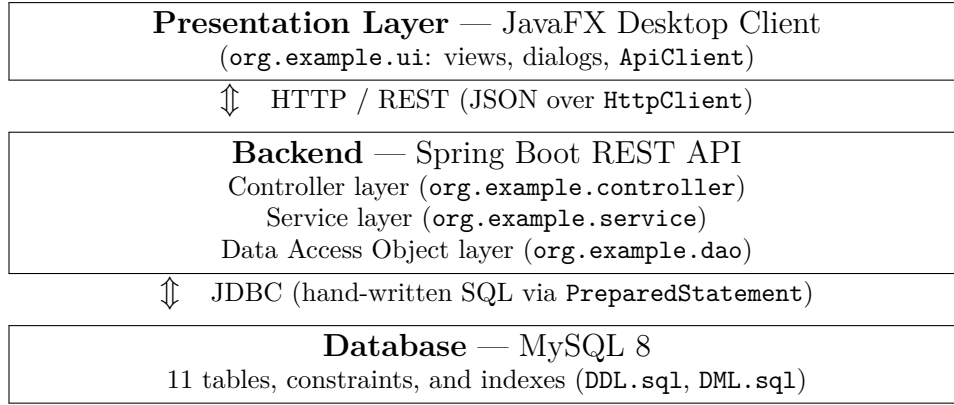


Figure 1: The three-tier architecture of the system.

- **Presentation layer.** A JavaFX desktop client renders all screens, captures user input, and never talks to the database directly. It reaches the backend exclusively through a thin HTTP client (`ApiClient`), which serializes requests to JSON and raises an exception on any non-2xx response.
- **Backend.** A Spring Boot application exposes the system’s capabilities as a Representational State Transfer (REST) interface. It is internally subdivided into three sub-layers: *controllers*, which receive HTTP requests, validate their structure, and return responses; *services*, which enforce every business rule and own transaction boundaries; and *Data Access Objects* (DAOs), which contain all SQL and map result sets to plain Java objects.
- **Database.** A MySQL 8 relational database stores all persistent data across 11 tables. The schema, its constraints, and its indexes are defined in `DDL.sql`; seed data is provided in `DML.sql`.

The backend is the single authoritative location for all business rules: because the user interface can only act through the REST API, there is no way for the client to bypass a rule such as the city restriction or the coupon validation.

1.3 Technology Stack

The implementation uses Spring Boot 3.2.5 [7] for the backend, JavaFX 21 [4] for the desktop client, and MySQL 8 [3] for storage. Communication between the database and the backend uses Java Database Connectivity (JDBC) [2] directly.

A central constraint of this project is that no Object-Relational Mapping (ORM) framework is used. Instead, every database operation is expressed as hand-written SQL inside the DAO layer and executed through a JDBC PreparedStatement. This choice keeps full control over the exact queries the application issues, makes the query design (threshold logic, relevance ranking, aggregate statistics) explicit and auditable, and directly demonstrates the database concepts that this project is meant to exercise. As a security consequence, the exclusive use of parameterized PreparedStatements means user-supplied values are always bound as data rather than concatenated into SQL text, which structurally prevents SQL-injection attacks.

1.4 Report Structure

The remainder of this report follows the structure of the design. Section 2 presents the Entity–Relationship (ER) diagram and walks through the entities and their relationships. Section 3 lists the functional dependencies of each table. Section 4 justifies the normalization of the schema up to Third Normal Form (3NF). Section 5 explains the key design decisions and integrity constraints. Section 6 describes the main SQL queries that drive the application, quoting the actual source. Section 7 documents the implemented functionalities from the perspective of each role. Section 8 concludes.

2 Entity–Relationship Diagram

The conceptual design of the database is captured by the Entity–Relationship (ER) diagram in Figure 2. The diagram is drawn in Chen’s notation [1]

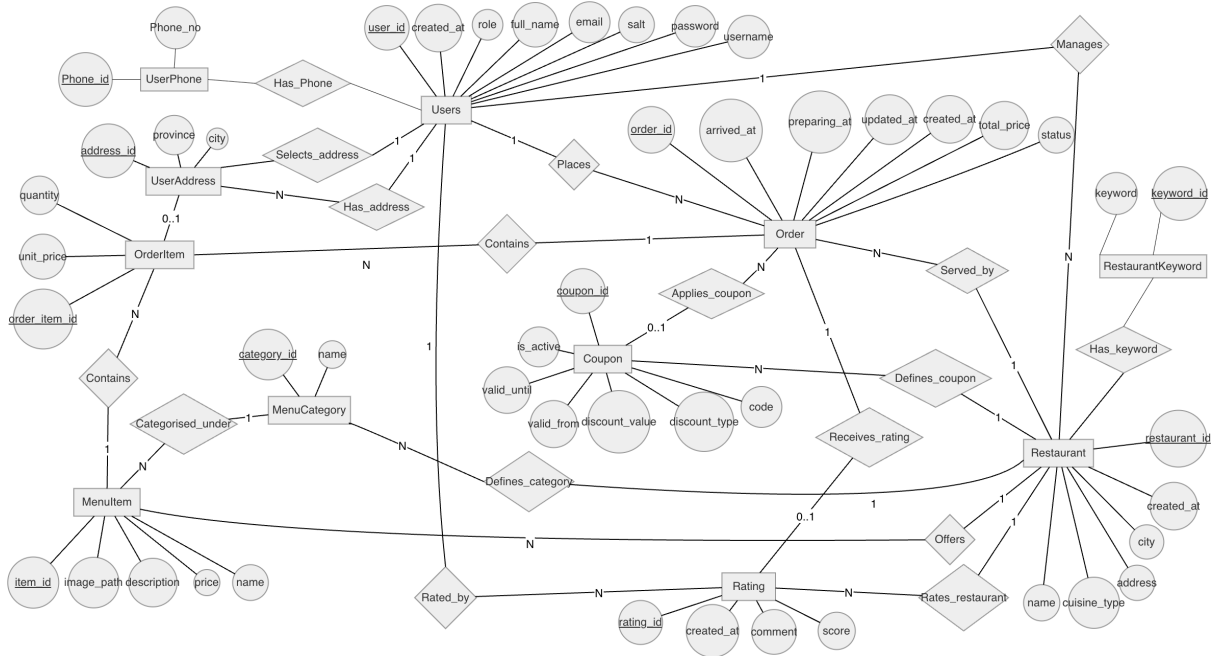


Figure 2: Entity–Relationship diagram of the Online Food Ordering System.

2.1 Entities

The design comprises eleven entity sets:

- **Users** — a single identity entity that represents both customers and managers, distinguished by a `role` attribute. We considered modeling this as an ISA (generalization) hierarchy with a supertype `User` and two subtypes `Customer` and `Manager`, but the two roles share almost all of their attributes (username, email, password, salt, full name) and differ only in which relationships they participate in. We therefore chose a single role-discriminated entity: it avoids duplicating the shared identity attributes across two tables, keeps login and authentication uniform, and lets a single foreign key reference a user regardless of role. The few role-specific rules (only a manager owns restaurants, only a customer places orders) are enforced in the service layer rather than by the schema shape.

- **UserAddress** — a (city, province) delivery address that belongs to one user; a user may hold many addresses and marks one as selected.
- **UserPhone** — a phone number belonging to one user; a user may hold many phone numbers.
- **Restaurant** — a food business owned by one manager, with a name, cuisine type, street address, and city.
- **RestaurantKeyword** — a single searchable tag attached to a restaurant; a restaurant may have many keywords.
- **MenuCategory** — a named grouping of menu items within one restaurant (for example “Starters” or “Main Dishes”).
- **MenuItem** — an individual dish with a price, description, and image, belonging to one restaurant and one category.
- **Coupon** — a discount code created by a manager for one of their restaurants, with a discount type, value, validity window, and active flag.
- **Order** — a customer’s purchase event from one restaurant at a point in time, with a status, a final price, and an optional coupon.
- **OrderItem** — one line within an order: a menu item, a quantity, and the unit price charged at order time.
- **Rating** — a customer’s 1–5 star review tied to one completed order.

Several of these are *dependent* (weak) entities that have no meaning without their owner and participate totally in an identifying relationship: **UserAddress** and **UserPhone** depend on **Users**, **RestaurantKeyword**, **MenuCategory**, and **MenuItem** depend on **Restaurant**, and **OrderItem** depends on **Order**. In the relational schema each is given its own surrogate key for compact references, but conceptually each is existence-dependent on its parent — which is exactly why the corresponding foreign keys use ON DELETE CASCADE (Section 5).

2.2 Relationships and Cardinalities

Table 1 summarizes the relationship sets and their cardinalities. Alongside the familiar $1:N$ ratio we also give the explicit (min, max) participation pairs in the notation taught in class: the first pair is the participation of the left-hand entity named in the **Relationship** column and the second pair that of the right-hand entity, with N denoting “many.” A minimum of 0 marks optional participation and a minimum of 1 marks total (mandatory) participation.

Relationship	Ratio	(min, max)	Meaning
Users <i>has</i> UserAddress	1 : N	(0,N), (1,1)	A user owns many addresses; each address belongs to one user.
Users <i>selects</i> UserAddress	1 : 1 (opt.)	(0,1), (0,1)	A user marks at most one address as selected for browsing and ordering.
Users <i>has</i> UserPhone	1 : N	(0,N), (1,1)	A user owns many phone numbers.
Users <i>manages</i> Restaurant	1 : N	(0,N), (1,1)	A manager owns many restaurants; each restaurant has one manager.
Restaurant <i>has</i> RestaurantKeyword	1 : N	(0,N), (1,1)	A restaurant has many search keywords.
Restaurant <i>has</i> MenuCategory	1 : N	(0,N), (1,1)	A restaurant has many menu categories.
Restaurant <i>has</i> MenuItem	1 : N	(0,N), (1,1)	A restaurant has many menu items.
MenuCategory <i>groups</i> MenuItem	1 : N	(0,N), (1,1)	A category contains many items; each item is in one category.
Restaurant <i>offers</i> Coupon	1 : N	(0,N), (1,1)	A restaurant has many coupons.
Users <i>places</i> Order	1 : N	(0,N), (1,1)	A customer places many orders.
Restaurant <i>receives</i> Order	1 : N	(0,N), (1,1)	A restaurant receives many orders.
Coupon <i>applied to</i> Order	1 : N (opt.)	(0,N), (0,1)	A coupon may be used on many orders; an order uses at most one coupon.
Order <i>contains</i> OrderItem	1 : N	(1,N), (1,1)	An order contains many line items; an item line belongs to one order.
MenuItem <i>appears in</i> OrderItem	1 : N	(0,N), (1,1)	A menu item appears in many order lines.
Order <i>rated by</i> Rating	1 : 1 (opt.)	(0,1), (1,1)	An order has at most one rating; a rating describes one order.
Users <i>writes</i> Rating	1 : N	(0,N), (1,1)	A customer writes many ratings.
Restaurant <i>receives</i> Rating	1 : N	(0,N), (1,1)	A restaurant receives many ratings.

Table 1: Relationship sets, cardinality ratios, and (min, max) participation constraints.

2.3 Note on a Design Revision

The schema went through one notable revision during design, which is worth recording because it directly improved the normalization of the model. In an earlier draft, the customer’s **city** was an attribute of the **Users** entity. This was problematic for two reasons: it limited a user to a single city, and it introduced a transitive dependency, because the city the user orders from is really an attribute of their *selected address*, not of the user identity itself. We therefore moved **city** (together with **province**) out of **Users** into the **UserAddress** entity and added a **selected_address_id** reference on **Users**. A user can now hold several addresses across different cities and switch the active one, and the schema satisfies Third Normal Form (the city is no longer transitively dependent on the user through the selected address). The order lifecycle was likewise finalized as the three-state progression **SENT** → **PREPARING** → **ARRIVED**, with dedicated **preparing_at** and **arrived_at** timestamps that record each transition.

3 Functional Dependencies

A functional dependency (FD) $X \rightarrow Y$ states that knowing the value of the attribute set X uniquely determines the value of Y . This section lists the functional dependencies of each table. In every table the surrogate primary key determines all other attributes; additional FDs arise from the `UNIQUE` constraints, and a few non-dependencies are noted explicitly because they motivate the normalization choices in Section 4.

3.1 Users

```
user_id → username, password, salt, email, full_name,  
        role, selected_address_id, created_at  
username → user_id, ... (username is UNIQUE)  
email → user_id, ... (email is UNIQUE)
```

Because both `username` and `email` are declared `UNIQUE`, each is a candidate key and therefore determines every other attribute of the row.

3.2 UserAddress

```
address_id → user_id, city, province
```

3.3 UserPhone

```
phone_id → user_id, phone
```

3.4 Restaurant

```
restaurant_id → manager_id, name, cuisine_type,  
               address, city, created_at  
manager_id ↗ restaurant_id
```

The non-dependency `manager_id ↗ restaurant_id` holds because one manager may own several restaurants, so the manager identifies a *set* of restaurants, not a single one. Note that `restaurant_id → city` is a genuine, direct dependency: the city is an intrinsic attribute of the restaurant itself, which is why `city` correctly lives on this table.

3.5 RestaurantKeyword

```
keyword_id → restaurant_id, keyword
```

3.6 MenuCategory

```
category_id → restaurant_id, name
```

3.7 MenuItem

$\text{item_id} \rightarrow \text{restaurant_id}, \text{category_id}, \text{name},$
 $\text{description}, \text{price}, \text{image_path}$

3.8 Coupon

$\text{coupon_id} \rightarrow \text{restaurant_id}, \text{code}, \text{discount_type},$
 $\text{discount_value}, \text{valid_from}, \text{valid_until}, \text{is_active}$
 $\text{code} \rightarrow \text{coupon_id}, \dots \quad (\text{code is UNIQUE})$

3.9 Order

$\text{order_id} \rightarrow \text{customer_id}, \text{restaurant_id}, \text{coupon_id}, \text{status}, \text{total_price},$
 $\text{created_at}, \text{updated_at}, \text{preparing_at}, \text{arrived_at}$
 $\text{order_id} \not\rightarrow \text{restaurant_name}$

The order has exactly one status, one final price (snapshotted at order time), and one delivery timestamp (or NULL until delivered), so each of these is determined by `order_id`. The non-dependency $\text{order_id} \not\rightarrow \text{restaurant_name}$ is deliberate: a restaurant's name is determined by `restaurant_id`, not by the order, so it is not stored on the `Order` table (see Section 4).

3.10 OrderItem

$\text{order_item_id} \rightarrow \text{order_id}, \text{item_id}, \text{quantity}, \text{unit_price}$
 $(\text{order_id}, \text{item_id}) \rightarrow \text{quantity}, \text{unit_price}$
 $\text{order_item_id} \not\rightarrow \text{current_price}$

The composite $(\text{order_id}, \text{item_id})$ also determines the quantity and unit price for that line. Note that `order_item_id` does *not* determine the item's current live price (`MenuItem.price`), because that price may change after the order is placed — precisely why `unit_price` is stored as a snapshot here.

3.11 Rating

$\text{rating_id} \rightarrow \text{customer_id}, \text{restaurant_id}, \text{order_id},$
 $\text{score}, \text{comment}, \text{created_at}$
 $\text{order_id} \rightarrow \text{rating_id} \quad (\text{order_id is UNIQUE})$

Because `order_id` is UNIQUE in `Rating`, it determines the rating row; this is exactly the constraint that enforces the business rule of at most one rating per order.

4 Normalization

The schema is normalized up to Third Normal Form (3NF) [5]. This section justifies each normal form against the actual tables, with concrete examples drawn from the design.

4.1 First Normal Form (1NF)

A relation is in 1NF if every attribute holds a single, atomic value — no repeating groups, comma-separated lists, or arrays. Two multi-valued attributes in the problem domain would otherwise tempt a 1NF violation, and both are resolved by extracting them into their own tables:

- **Restaurant keywords.** We do *not* store `keywords = "turkish,kebab,grilled"` in a single column on `Restaurant`. Each keyword is one row in `RestaurantKeyword`, linked by `restaurant_id`. Besides satisfying 1NF, this is what allows the search query to match an individual keyword cleanly with `LIKE` and to count distinct keyword matches for relevance ranking.
- **User phone numbers and addresses.** We do not store multiple phone numbers or addresses as a delimited string on `Users`. Each phone number is a row in `UserPhone`, and each address is a row in `UserAddress`.

Every column in every table therefore holds exactly one atomic value, so all eleven tables are in 1NF.

4.2 Second Normal Form (2NF)

A relation is in 2NF if it is in 1NF and has no partial dependencies — that is, no non-key attribute depends on only part of a composite primary key. Partial dependencies can only arise when a table has a *composite* primary key.

In our schema, every table uses a single-column surrogate primary key (`INT AUTO_INCREMENT`). A single-column key has no proper subset to be “partially” dependent on, so 2NF is satisfied trivially by all eleven tables. For example, in `OrderItem`, even though the combination (`order_id`, `item_id`) functionally determines `quantity` and `unit_price`, the actual primary key is the surrogate `order_item_id`; both non-key attributes depend on the whole key, so there is no partial dependency.

4.3 Third Normal Form (3NF)

A relation is in 3NF if it is in 2NF and has no transitive dependencies — that is, no non-key attribute determines another non-key attribute. We illustrate this with two worked examples that directly shaped the schema.

Example 1 — why `city` is not stored on `Users`. Suppose we placed a `city` column on `Users` while also keeping `selected_address_id`. We would then have:

$$\text{user_id} \rightarrow \text{selected_address_id} \rightarrow \text{city}.$$

This is a transitive dependency: `user_id` determines `city` only *through* the non-key attribute `selected_address_id`. The city is an attribute of the selected address, not of the

user identity. Keeping it on **Users** would duplicate the value and cause update anomalies — if a user edited their address city, both `UserAddress.city` and `Users.city` would have to be updated in lock-step. The schema therefore stores `city` only on `UserAddress`, and the application derives the active city by joining through `selected_address_id` when it is needed.

Example 2 — why `restaurant_name` is not stored on `Order`. Suppose we stored `restaurant_name` on the `Order` table. Then:

$$\text{order_id} \rightarrow \text{restaurant_id} \rightarrow \text{restaurant_name}.$$

Again the dependency is transitive: the name is determined by `restaurant_id`, a non-key attribute. Storing the name on `Order` would create an update anomaly — renaming a restaurant would leave every historical order showing the old name. Instead, the order stores only `restaurant_id`, and any view that needs the name joins through it.

The same reasoning excludes other derivable values (item names, category names, manager names) from the tables that merely reference them. No non-key attribute in any table determines another non-key attribute, so all eleven tables are in 3NF.

4.4 Deliberate, Documented Denormalizations

Two values in the schema are stored even though they could, at first glance, be recomputed. These are not normalization violations but intentional *snapshots* required for historical correctness:

- `OrderItem.unit_price` records the price charged for a line at the moment of purchase. It is not a transitive copy of `MenuItem.price`, because the live menu price can change later; the order must permanently reflect what the customer actually paid.
- `Order.total_price` records the final, post-coupon total paid. It is read directly as confirmed revenue by the statistics module. Recomputing it from current prices would silently rewrite historical figures whenever a menu price changed.

Storing these snapshots is a correctness requirement, not redundancy: the stored value depends on the order event itself (the price *at that time*), which is information that no other table preserves. Similarly, `Rating` stores both `customer_id` and `restaurant_id` directly even though the latter could be reached through `order_id`; this is a deliberate choice to keep the common “ratings for a restaurant” query a single indexed lookup rather than an extra join, and each remains a genuine attribute of the rating event.

5 Key Design Decisions and Constraints

This section explains the main design decisions and the integrity constraints that enforce the system’s business rules. Constraints are placed at the database level wherever the database can express them, and reinforced in the service layer where a rule spans multiple tables or requires application logic.

5.1 Storage Engine, Initialization, and Sample Data

The schema is defined by two scripts: `DDL.sql` creates the database and its eleven tables with all constraints, and `DML.sql` loads a representative set of sample data. All tables use MySQL 8’s default `InnoDB` storage engine, which is precisely what makes the foreign-key actions and the transactional guarantees described in this section enforceable — the legacy `MyISAM` engine, by contrast, parses foreign-key clauses but silently ignores them and offers no transactions. For a reproducible, “clone-and-run” setup, a `docker-compose` service runs the `mysql:8.0` image and mounts the two scripts into the container’s initialization directory as `01-DDL.sql` and `02-DML.sql`, so a fresh container builds the schema and then seeds it automatically, in that order, on first start. The seed data covers eight users (three managers and five customers) with their addresses and phone numbers, several restaurants with search keywords, menu categories and items, coupons, completed orders with their line items, and ratings — enough to exercise browsing, search, ordering, the full lifecycle, and the monthly statistics immediately after startup.

5.2 Primary, Unique, and Not-Null Constraints

Every table has a single-column surrogate primary key (`INT AUTO_INCREMENT`). A surrogate key keeps foreign-key references compact and stable, and frees natural keys (such as a username) to change without rewriting references. Beyond the primary keys, several `UNIQUE` constraints enforce real business rules: `Users.username` and `Users.email` guarantee that no two accounts share a login handle or an email; `Coupon.code` guarantees system-wide uniqueness of coupon codes; and `Rating.order_id` guarantees at most one rating per order. Mandatory columns are declared `NOT NULL`, while genuinely optional references — `Users.selected_address_id`, `Order.coupon_id`, and `Rating.comment` — are left nullable.

5.3 ENUM and CHECK Constraints

Closed value sets are modeled with `ENUM` so that the database itself rejects illegal values: `Users.role` is restricted to `CUSTOMER` or `MANAGER`; `Order.status` to `SENT`, `PREPARING`, or `ARRIVED`; and `Coupon.discount_type` to `PERCENTAGE` or `AMOUNT`. Using `ENUM` rather than a free-text column means a misspelling such as `ARRIVD` can never be persisted and silently break the status comparisons throughout the application.

`CHECK` constraints enforce numeric and temporal invariants: `MenuItem.price > 0` and `Coupon.discount_value > 0` forbid free or negative-priced items and discounts; `OrderItem.quantity > 0` and `OrderItem.unit_price > 0` keep line items meaningful; `Order.total_price >= 0` permits a zero total (a 100% coupon) but never a negative one; `Coupon.valid_until >= valid_from` prevents an expiry that precedes the start date; and `Rating.score BETWEEN 1 AND 5` pins the rating scale at the database level.

5.4 Foreign Keys and ON DELETE Behavior

Each foreign key chooses an `ON DELETE` action that encodes a specific business rule, summarized in Table 2.

Foreign key	ON DELETE	Business rule enforced
UserAddress.user_id	CASCADE	Deleting a user removes their addresses (no orphan addresses).
UserPhone.user_id	CASCADE	Deleting a user removes their phone numbers.
Users.selected_address_id	SET NULL	Deleting the selected address clears the pointer but keeps the account.
Restaurant.manager_id	RESTRICT	A manager who still owns restaurants cannot be deleted.
RestaurantKeyword.restaurant_id	CASCADE	Deleting a restaurant removes its keywords.
MenuCategory.restaurant_id	CASCADE	Deleting a restaurant removes its categories.
MenuItem.restaurant_id	CASCADE	Deleting a restaurant removes its items.
MenuItem (restaurant_id, category_id)	RESTRICT	A category that still has items cannot be deleted.
Coupon.restaurant_id	CASCADE	Deleting a restaurant removes its coupons.
Order.customer_id	RESTRICT	A customer with orders cannot be deleted (orders are permanent records).
Order.restaurant_id	RESTRICT	A restaurant with orders cannot be deleted.
Order.coupon_id	SET NULL	A removed coupon leaves the historical order intact, without the reference.
OrderItem.order_id	CASCADE	Deleting an order removes its line items.
OrderItem.item_id	RESTRICT	A menu item that appears in a historical order cannot be deleted.
Rating.customer_id / restaurant_id / order_id	CASCADE	A rating cannot outlive the entity it describes.

Table 2: Foreign-key delete behavior and the rule each enforces.

The pattern is consistent: **CASCADE** is used where a child has no meaning without its parent (an address without its user, a keyword without its restaurant); **RESTRICT** protects historical and ownership data (orders, owned restaurants, items that appear in past orders); and **SET NULL** preserves a record while clearing an optional reference (a selected address, an applied coupon).

5.5 The Circular Users ↔ UserAddress Reference

UserAddress.user_id references Users, while Users.selected_address_id references UserAddress — a circular foreign-key dependency that cannot be satisfied if both tables are created with all constraints inline. We resolve this by creating Users with selected_address_id nullable and *no* foreign key, creating UserAddress, and then adding the missing constraint with a deferred ALTER TABLE (DDL.sql lines 43–46) using ON DELETE SET NULL. This keeps the creation order valid for the dockerized MySQL 8 initialization and keeps inserts and deletes orderable: a new user can be inserted before any address exists, and deleting the selected address simply clears the pointer.

5.6 Composite Foreign Key on MenuItem

MenuItem references MenuCategory through the *composite* foreign key (restaurant_id, category_id), backed by a matching UNIQUE key on MenuCategory(restaurant_id, category_id). This guarantees at the database level that an item can only be filed under a category that belongs to the *same* restaurant — it is structurally impossible to assign an item to another restaurant’s category.

5.7 Application-Level Business Rules

Some rules cannot be expressed as a single-table constraint and are enforced in the service layer, which is the authoritative location for all business logic:

- **Manager-owns-restaurant.** A MySQL CHECK cannot reference another table to verify that a restaurant's `manager_id` actually has the `MANAGER` role, so the `chk_manager_role` constraint is a placeholder and the real check lives in the service layer, in the `requireOwnership` method of `RestaurantService`.
- **City restriction (defense in depth).** A customer may only order from a restaurant in the city of their selected address. This is enforced in layers: the browse and search queries filter by `r.city = ?`, the UI will not offer out-of-city restaurants, and the service method `placeOrder` re-checks the customer's city against the restaurant's city (case-insensitively) before writing the order, rejecting a mismatch with an error that maps to HTTP 400.
- **Price snapshots from the database.** At checkout, `placeOrder()` ignores any client-supplied price and fetches the current price from the database for each item, storing it as `OrderItem.unit_price`. This prevents price tampering and produces the historical snapshots discussed in Section 4.
- **Guarded, atomic state transitions.** Order status changes use guarded `UPDATE`s with an `AND status = ...` clause so a transition only succeeds from the expected state (see Section 6.4). The service treats an affected-row count of zero as an illegal transition and raises a conflict (HTTP 409).
- **One rating per order, within 24 hours.** In addition to the `UNIQUE(order_id)` constraint, the service verifies that the order is delivered, that no rating already exists, and that the request arrives within 24 hours of `arrived_at`.
- **Coupon soft-deactivation.** Managers deactivate coupons by clearing the `is_active` flag rather than deleting the row, so historical orders that reference the coupon are preserved. Validity is the conjunction of the active flag and the date window given by `valid_from` and `valid_until`.
- **Transactional order placement.** `placeOrder()` is annotated `@Transactional`: the order header and all its line items are written in one transaction, so a failure on any insert rolls the whole order back.

5.8 Password Security

Passwords are never stored in plain text. At registration, a 16-byte random salt is generated and the password is hashed with PBKDF2-HMAC-SHA256 (Password-Based Key Derivation Function 2 using a Hash-based Message Authentication Code over the 256-bit Secure Hash Algorithm) [6] at 100,000 iterations. The hash and salt are stored in `Users.password` and `Users.salt`. A per-user salt defeats precomputed rainbow-table attacks, the high iteration count slows brute-forcing, and verification uses a constant-time comparison to avoid leaking timing information.

5.9 Centralized Error Handling

All exceptions thrown anywhere in the controller or service chain are translated to HTTP status codes by a single `@RestControllerAdvice` handler: `IllegalArgumentException` and malformed-input exceptions map to 400 Bad Request, `IllegalStateException` maps to 409 Conflict, and any other exception maps to 500 Internal Server Error. The handler logs the cause but returns a clean message without exposing stack traces, keeping error handling consistent across every endpoint.

6 Main SQL Queries

Every database operation in the system is hand-written SQL executed through a JDBC `PreparedStatement`; there is no ORM and no string concatenation into SQL anywhere in the codebase. This section presents the most important queries that drive the application, quoted from the actual source, with an explanation of why each is written the way it is.

6.1 Restaurant Search with Relevance Ranking

Source: `RestaurantDAO.searchByKeywordAndCity()`.

```
SELECT r.*,
       CASE WHEN COUNT(DISTINCT rt.rating_id) >= 10 THEN AVG(rt.score)
            ELSE 0 END AS avg_rating,
       COUNT(DISTINCT rt.rating_id) AS rating_count
FROM Restaurant r
JOIN RestaurantKeyword rk ON r.restaurant_id = rk.restaurant_id
LEFT JOIN Rating rt ON r.restaurant_id = rt.restaurant_id
WHERE rk.keyword LIKE ? AND r.city = ?
GROUP BY r.restaurant_id
ORDER BY COUNT(DISTINCT rk.keyword) DESC,
         CASE WHEN COUNT(DISTINCT rt.rating_id) >= 10 THEN
             AVG(rt.score) ELSE 0 END DESC,
         r.name ASC
```

This single query implements search, the city restriction, the rating-display rule, and relevance ranking together:

- The JOIN to `RestaurantKeyword` is an *inner* join, so a restaurant with no matching keyword is excluded from search results. The `LEFT JOIN` to `Rating` is an *outer* join, so a restaurant with zero ratings still appears.
- `WHERE rk.keyword LIKE ?` is bound to `"%" + keyword + "%"`, and `r.city = ?` enforces the city restriction.
- The `CASE WHEN COUNT(DISTINCT rt.rating_id) >= 10` expression implements the threshold rule: a restaurant with fewer than ten ratings reports an average of 0 (the UI then labels it “New”), so a single five-star review cannot rank a brand-new restaurant above an established one.
- `COUNT(DISTINCT ...)` is required because a restaurant matching several keywords produces several joined rows; `DISTINCT` prevents overcounting ratings.

- The `ORDER BY` ranks by the number of distinct matching keywords first (relevance), then by the thresholded rating, then alphabetically as a deterministic tiebreaker.

6.2 City-Filtered Browse

Source: `RestaurantDAO.findByCity()`.

```
SELECT r.*,
       CASE WHEN COUNT(rt.rating_id) >= 10 THEN AVG(rt.score) ELSE 0 END
       AS avg_rating,
       COUNT(rt.rating_id) AS rating_count
FROM Restaurant r
LEFT JOIN Rating rt ON r.restaurant_id = rt.restaurant_id
WHERE r.city = ?
GROUP BY r.restaurant_id
ORDER BY CASE WHEN COUNT(rt.rating_id) >= 10 THEN AVG(rt.score) ELSE
0 END DESC, r.name ASC
```

When the customer browses without a keyword, there is no need to join `RestaurantKeyword` at all; doing so via an inner join would wrongly hide restaurants that have no keywords. This query therefore omits that join, drops the relevance term from the ordering, and (because there are no duplicate keyword rows) does not need `DISTINCT` on the count.

6.3 Order Placement (Two Inserts)

Source: the `insert` and `insertItem` methods of `OrderDAO`, orchestrated by `placeOrder` in `OrderService`.

```
-- 1. The order header; the generated order_id is retrieved
    afterwards.
INSERT INTO 'Order' (customer_id, restaurant_id, coupon_id, status,
    total_price)
VALUES (?, ?, ?, ?, ?);

-- 2. One row per line item, using the order_id from step 1.
INSERT INTO OrderItem (order_id, item_id, quantity, unit_price)
VALUES (?, ?, ?, ?);
```

Because an order and its items live in separate tables (a consequence of normalization), placement is two statements: the header is inserted first with `Statement.RETURN_GENERATED_KEYS` so JDBC returns the new `order_id` without a second query, and that key becomes the foreign key for each `OrderItem` insert. The whole method is `@Transactional`: the service first fetches each item's current price from the database (ignoring any client-supplied value), computes the subtotal, applies a coupon if present, sets the status to `SENT`, and then performs the inserts — if any step fails, the entire order rolls back.

6.4 Guarded Status Transitions

Source: `OrderDAO.markPreparing()` / `markArrived()`.

```

-- SENT -> PREPARING (manager accepts the order)
UPDATE 'Order' SET status = 'PREPARING', preparing_at = NOW()
WHERE order_id = ? AND status = 'SENT';

-- PREPARING -> ARRIVED (manager marks the order delivered)
UPDATE 'Order' SET status = 'ARRIVED', arrived_at = NOW()
WHERE order_id = ? AND status = 'PREPARING';

```

The `AND status = ...` clause makes each update an atomic test-and-set: it affects one row only if the order is currently in the expected state. The service inspects the affected-row count and, if it is zero, raises an `IllegalStateException` (mapped to HTTP 409) — no separate `SELECT` is needed to check the current state first, which also avoids a race between two managers acting on the same order. The transition timestamp is set with MySQL’s `NOW()` so it uses the authoritative database clock; `arrived_at` specifically anchors the 24-hour rating window.

6.5 Monthly Statistics

Source: `StatisticsService`. The service computes seven metrics for a restaurant over the current calendar month. Each query filters on `status = 'ARRIVED'` (only confirmed, delivered orders count as revenue) and appends a month-window fragment that expands to:

```

AND YEAR(created_at) = YEAR(CURDATE())
AND MONTH(created_at) = MONTH(CURDATE())

```

Using `CURDATE()` means “the month in which the report is run”, so the summary always describes the live current month. The total-revenue metric is a direct aggregate over the snapshot totals:

```

SELECT COALESCE(SUM(total_price), 0)
FROM 'Order'
WHERE restaurant_id = ? AND status = 'ARRIVED'
  AND YEAR(created_at) = YEAR(CURDATE())
  AND MONTH(created_at) = MONTH(CURDATE());

```

`COALESCE(..., 0)` returns 0 rather than `NULL` when a restaurant has no qualifying orders. “Top customer by order count” uses the standard *group-and-rank* idiom:

```

SELECT u.user_id, u.full_name, COUNT(*) AS order_count
FROM 'Order' o JOIN Users u ON o.customer_id = u.user_id
WHERE o.restaurant_id = ? AND o.status = 'ARRIVED'
  AND YEAR(o.created_at) = YEAR(CURDATE())
  AND MONTH(o.created_at) = MONTH(CURDATE())
GROUP BY o.customer_id, u.user_id, u.full_name
ORDER BY order_count DESC, u.user_id ASC
LIMIT 1;

```

`GROUP BY` collapses each customer’s orders into one row, `ORDER BY ...DESC` ranks them, and `LIMIT 1` takes the top; `u.user_id ASC` breaks ties deterministically. The “top revenue category” metric uses a four-table join to attribute snapshot revenue to a category:

```

SELECT mc.name, SUM(oi.quantity * oi.unit_price) AS cat_revenue
FROM OrderItem oi
JOIN MenuItem mi ON oi.item_id = mi.item_id
JOIN MenuCategory mc ON mi.category_id = mc.category_id
JOIN 'Order' o ON oi.order_id = o.order_id
WHERE o.restaurant_id = ? AND o.status = 'ARRIVED'
      AND YEAR(o.created_at) = YEAR(CURDATE())
      AND MONTH(o.created_at) = MONTH(CURDATE())
GROUP BY mc.category_id, mc.name
ORDER BY cat_revenue DESC, mc.category_id ASC
LIMIT 1;

```

The chain starts at `OrderItem` (where `quantity` and `unit_price` live), joins up to `MenuCategory` for the category name, and joins `Order` to apply the restaurant, status, and month filters. Revenue is computed from the snapshot `unit_price`, not the current menu price, so historical figures are stable.

The *per-item quantity and revenue* breakdown — the table shown on the statistics screen — groups the same three-table join by menu item instead of by category:

```

SELECT mi.name, SUM(oi.quantity) AS total_qty,
      SUM(oi.quantity * oi.unit_price) AS total_revenue
FROM OrderItem oi
JOIN MenuItem mi ON oi.item_id = mi.item_id
JOIN 'Order' o ON oi.order_id = o.order_id
WHERE o.restaurant_id = ? AND o.status = 'ARRIVED'
      AND YEAR(o.created_at) = YEAR(CURDATE())
      AND MONTH(o.created_at) = MONTH(CURDATE())
GROUP BY mi.item_id, mi.name
ORDER BY total_revenue DESC;

```

The *total coupon discount* is the one metric that requires a derived table. For each delivered order that used a coupon, a sub-select aggregates the order's line items into a pre-coupon `item_sum`; joining that back to `Order` lets the outer query subtract the stored post-coupon `total_price`, and the differences are summed across all such orders:

```

SELECT COALESCE(SUM(item_sum - o.total_price), 0)
FROM 'Order' o
JOIN (SELECT order_id, SUM(quantity * unit_price) AS item_sum
      FROM OrderItem GROUP BY order_id) s ON s.order_id = o.order_id
WHERE o.restaurant_id = ? AND o.status = 'ARRIVED'
      AND o.coupon_id IS NOT NULL
      AND YEAR(o.created_at) = YEAR(CURDATE())
      AND MONTH(o.created_at) = MONTH(CURDATE());

```

Because `total_price` is the post-coupon snapshot, the difference is exactly the amount the coupon removed. The three remaining metrics not quoted here — the order count, the most-ordered item, and the highest-value order with its line items — reuse the same patterns already shown (a `COUNT`, a quantity-ranked `GROUP BY ...LIMIT 1`, and a `total_price`-ranked `LIMIT 1`), so all seven metrics are accounted for.

6.6 Coupon Resolution

Source: `CouponService.resolveForOrder()`, backed by `CouponDAO.findByCode()`.

```
SELECT * FROM Coupon WHERE code = ?;
```

The looked-up coupon then passes four checks before any discount applies: existence, restaurant match (a coupon is only valid at its own restaurant), validity (the `is_active` flag *and* the current date within `valid_from/valid_until`), and finally the discount computation. A `PERCENTAGE` coupon multiplies the subtotal by $(1 - \text{pct}/100)$ with the percentage clamped to $[0, 100]$; an `AMOUNT` coupon subtracts a flat value, clamped so the total never drops below zero.

6.7 Login and Salt Retrieval

Source: `UserService.login()`, backed by `UserDAO`.

```
SELECT salt FROM Users WHERE username = ?;    -- fetch the stored
per-user salt
SELECT * FROM Users WHERE username = ?;        -- fetch the full row
(incl. hash)
```

Login fetches the stored salt, re-hashes the submitted password with it using PBKDF2-HMAC-SHA256, and compares the result to the stored hash using a constant-time comparison. Because each user has a unique salt, two users with the same password still have different stored hashes.

6.8 Parameterized Statements and Injection Prevention

Source: representative of every DAO, e.g. `UserDAO.findByUsername()`.

```
String sql = "SELECT * FROM Users WHERE username = ?";
PreparedStatement ps = conn.prepareStatement(sql);
ps.setString(1, username);
ResultSet rs = ps.executeQuery();
```

The `?` placeholder is a bound parameter: the database compiles the query with the parameter unknown, and the supplied value is sent separately and treated purely as data. A malicious username such as `' OR '1'='1` is matched literally against the column rather than altering the query structure. Had the value been concatenated into the SQL string, that same input would have turned into an always-true predicate returning every user — a classic SQL-injection attack. Every query in every DAO is written this way, so the application has no SQL-injection surface.

7 Implemented Functionalities

This section documents the functionality implemented in the system, organized by the two user roles and the system-wide services that support them. Each feature is described in prose; a screenshot slot is provided alongside so that the corresponding screen from the running JavaFX client can be inserted.

7.1 Account and Session Management

Registration. A new user registers by choosing a username, email, full name, password, and role, and by entering the city and province of a first delivery address. The backend hashes the password with a freshly generated salt and creates the user together with the first address in a single atomic operation, then marks that address as selected.

Login. A user logs in with username and password. On success the client routes to the customer dashboard or the manager dashboard according to the user’s role.

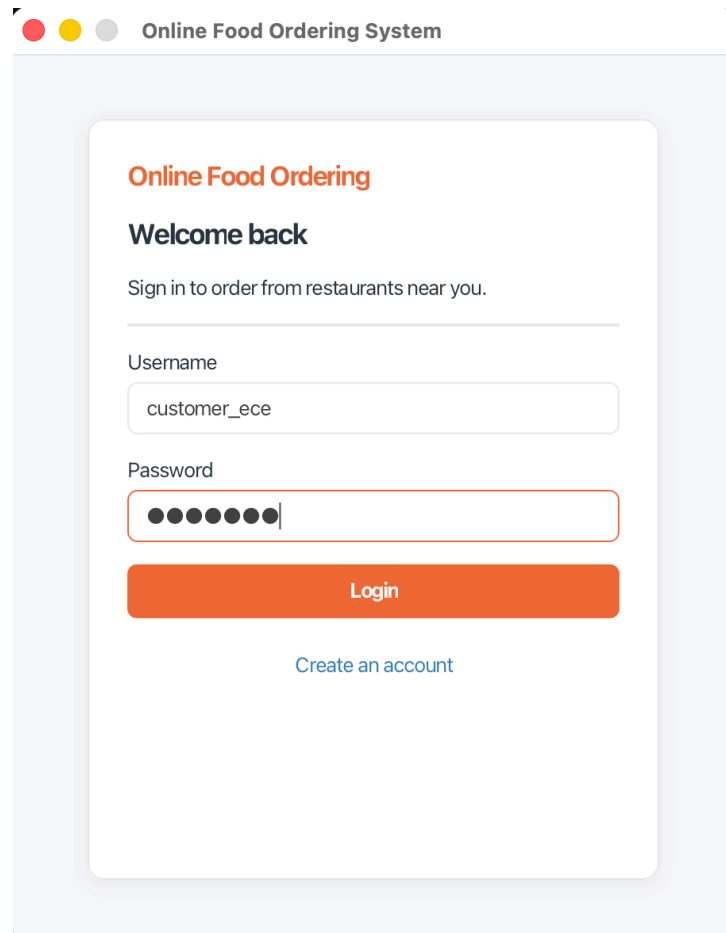
The image shows a web browser window titled "Online Food Ordering System". Inside the browser, there is a white card with a light orange border. At the top of the card, it says "Online Food Ordering" in orange, followed by "Welcome back" in bold black. Below this is the text "Sign in to order from restaurants near you." and a horizontal line. There are two input fields: "Username" with the text "customer_ece" and "Password" with masked characters (dots). Below the password field is an orange "Login" button. At the bottom of the card is a blue link that says "Create an account".

Figure 3: Login and registration.

7.2 Customer Functionalities

Address and phone management. A customer can add, delete, and select among multiple delivery addresses, and add or remove phone numbers. The currently selected address determines the city used for browsing and ordering; changing the selected address changes which restaurants are visible.

Browsing and searching restaurants. The customer sees the restaurants in the city of their selected address, sorted by rating (restaurants with fewer than ten ratings are shown as “New”). A keyword search filters the same city-scoped list and ranks results by how many keywords match, then by rating.

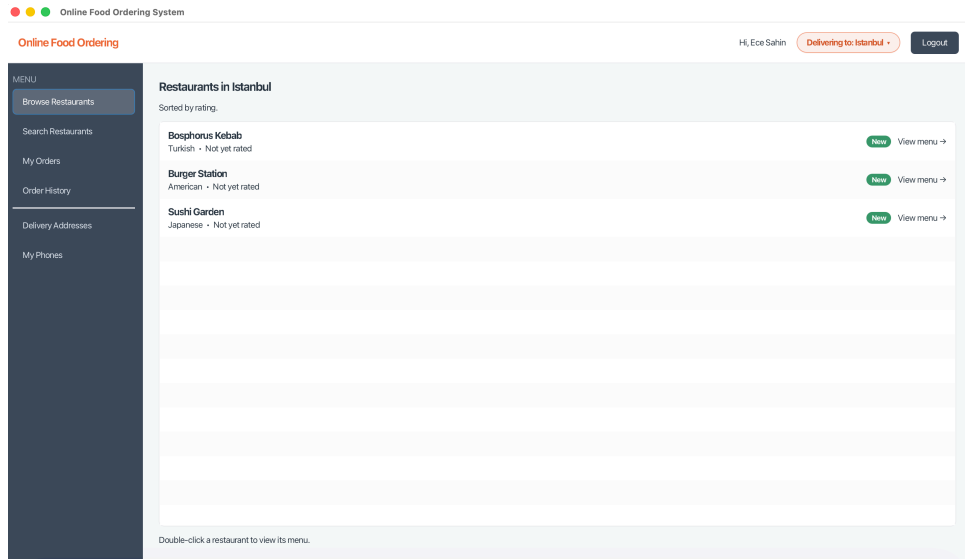


Figure 4: Browsing and searching restaurants in the selected city.

Viewing a menu and building a cart. Selecting a restaurant opens its menu grouped by category. The customer adds items and quantities to a cart, which tracks the running subtotal.

Applying a coupon and placing an order. At checkout the customer can preview a coupon (the “Check” action) and apply it. On placing the order, the backend validates the coupon, fetches current item prices from the database, computes the final total, and stores the order in **SENT** status together with its line items.

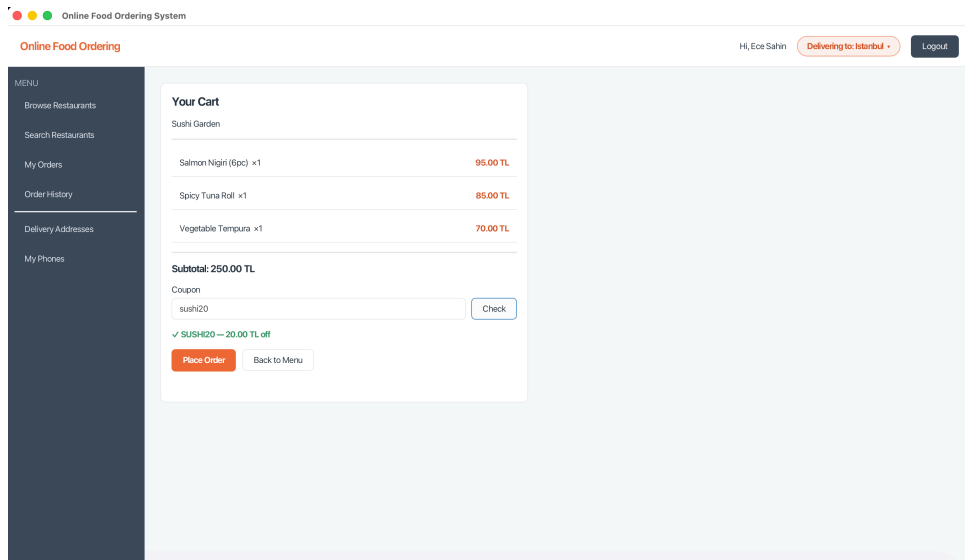


Figure 5: Cart, coupon entry, and order placement.

Tracking orders and viewing history. The customer can follow active orders as they advance through **SENT** → **PREPARING** → **ARRIVED**, and review completed (**ARRIVED**) orders in their history.

Rating a restaurant. Within 24 hours of delivery, the customer can leave a 1–5 star rating with an optional comment. The system enforces that the order is delivered, that no rating already exists for it, and that the 24-hour window has not elapsed.

Figure 6: Leaving a rating after delivery.

7.3 Manager Functionalities

Restaurant management. A manager creates restaurants and edits their details, including the searchable keyword list. All management actions are guarded by an ownership check so a manager can only modify their own restaurants.

Menu management. The manager organizes a restaurant’s menu into categories and adds, edits, and deletes menu items, each with a name, description, price, and image path. The composite foreign key guarantees an item can only be filed under a category of the same restaurant.

Figure 7: Managing categories and menu items.

Coupon management. The manager creates coupons (percentage or fixed amount, with a validity window) and deactivates existing ones. Deactivation clears the active flag rather than deleting the coupon, preserving the historical orders that used it.

Processing incoming orders. The manager sees incoming SENT and PREPARING orders, accepts an order (SENT $\rightarrow$ PREPARING), and later marks it delivered (PREPARING $\rightarrow$ ARRIVED). Each transition is a guarded, atomic update that only succeeds from the correct prior state.

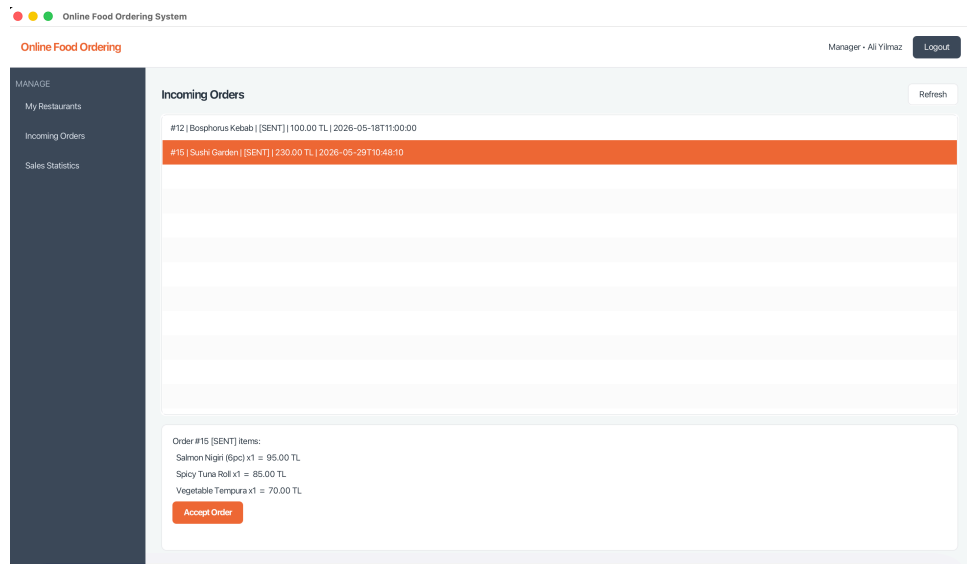


Figure 8: Processing incoming orders through the lifecycle.

Viewing ratings and monthly statistics. The manager can read the ratings left for their restaurants and open a monthly sales summary. The summary reports seven metrics over the current calendar month (restricted to delivered orders): total revenue together with the total order count, item-level revenue, the top customer by order count, the top customer by spend, the most-ordered item, the top revenue category, and the total discount granted through coupons.

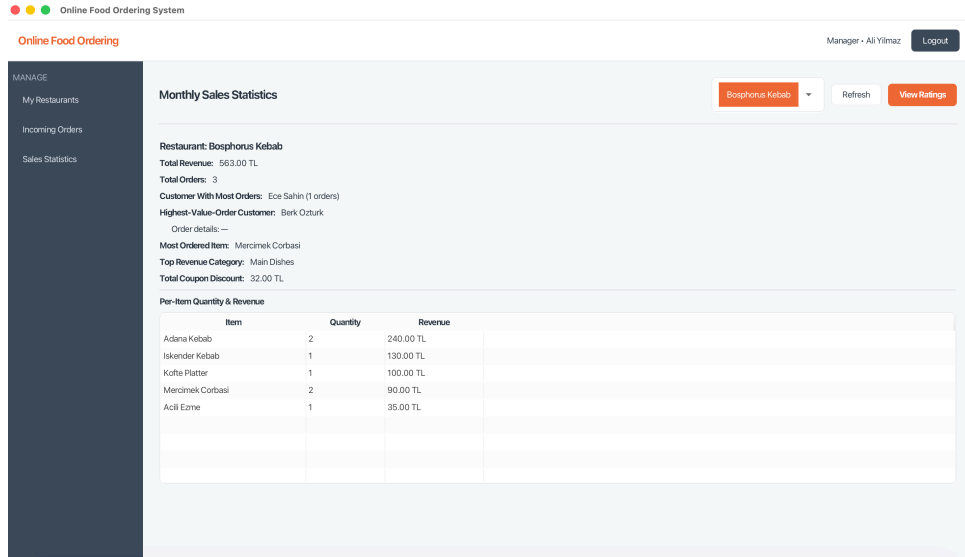


Figure 9: Monthly sales statistics for a restaurant.

7.4 System-Wide Services

Underpinning these features are the cross-cutting services described earlier: salted PBKDF2 password hashing for every account, the city-restriction guard applied on browsing and re-checked at order time, the price-snapshot rule that records what the customer actually paid, transactional order placement, and centralized error handling that maps every failure to a consistent HTTP status code. Together these ensure that the user-facing functionality above rests on a database and backend that enforce the system’s rules regardless of how the client behaves.

8 Conclusion

This report presented the design and implementation of an Online Food Ordering System built on a relational MySQL 8 database, a Spring Boot REST backend, and a JavaFX desktop client, with a strict three-layer separation between presentation, application logic, and data access. We described the Entity–Relationship model and its eleven entities, listed the functional dependencies of each table, and justified the normalization of the schema up to Third Normal Form, including two worked transitive-dependency examples and the deliberate, documented price snapshots that preserve historical correctness.

We then explained the key design decisions and the integrity constraints that enforce the system’s business rules — primary, unique, not-null, **ENUM**, and **CHECK** constraints at the database level; carefully chosen **ON DELETE** behaviors; the composite foreign key that scopes menu items to their restaurant’s categories; the deferred foreign key that resolves the circular **Users** / **UserAddress** reference; and the application-level rules (city restriction, guarded state transitions, the 24-hour rating window, coupon validation, and salted password hashing) that span multiple tables. The main SQL queries were presented directly from the source to show how search ranking, the rating threshold, the order lifecycle, and the monthly statistics are expressed in hand-written, parameterized SQL.

A defining characteristic of the project is that all database access is hand-written SQL over JDBC, executed exclusively through prepared statements, with no Object-Relational

Mapping framework. This kept full control over every query, made the database design explicit and auditable, and structurally eliminated the SQL-injection surface. The result is a complete, layered application whose business rules are enforced authoritatively at the backend and the database, independent of how the client is used.

References

- [1] Peter Pin-Shan Chen. The entity-relationship model—toward a unified view of data. *ACM Transactions on Database Systems*, 1(1):9–36, 1976.
- [2] Oracle Corporation. *Java Database Connectivity (JDBC) Guide*, 2014. JDBC API, Java SE 8. <https://docs.oracle.com/javase/8/docs/technotes/guides/jdbc/>.
- [3] Oracle Corporation. *MySQL 8.0 Reference Manual*, 2024. <https://dev.mysql.com/doc/refman/8.0/en/>.
- [4] Oracle Corporation / OpenJFX. *JavaFX 21 API Documentation*, 2023. <https://openjfx.io/>.
- [5] Abraham Silberschatz, Henry F. Korth, and S. Sudarshan. *Database System Concepts*. McGraw-Hill, seventh edition, 2019.
- [6] Meltem Sönmez Turan, Elaine Barker, William Burr, and Lily Chen. Recommendation for Password-Based Key Derivation, Part 1: Storage Applications. NIST Special Publication 800-132, National Institute of Standards and Technology, 2010. <https://csrc.nist.gov/publications/detail/sp/800-132/final>.
- [7] VMware, Inc. *Spring Boot Reference Documentation*, 2024. Version 3.2.x. <https://docs.spring.io/spring-boot/docs/current/reference/html/>.